Lab 04 - File I/O in Java

Student name: _____

Student ID: _____

Class / Group: _____

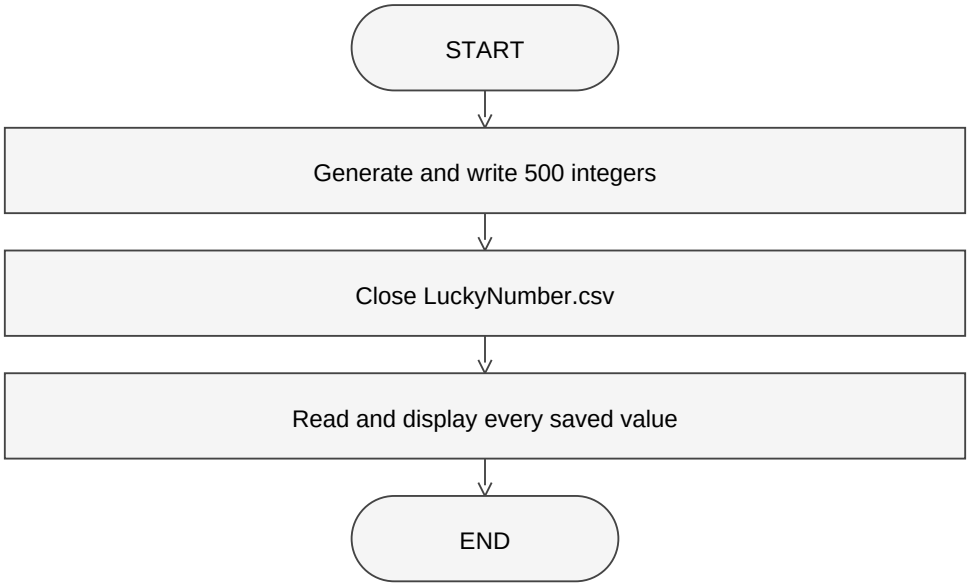
Exercise 1: Lucky numbers

Generate, save, and reread 500 integers.

Pseudocode

```
START
OPEN LuckyNumber.csv for writing
REPEAT 500 times
    GENERATE an integer from 1 to 100
    WRITE the integer to the file
END REPEAT
CLOSE the writer
READ and DISPLAY every saved integer
DISPLAY the count of 500 numbers
END
```

Flowchart



Exercise 1 - Source code

Exercise1.java

Exercise1.java

```java
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.Random;

public class Exercise1 {
    public static void main(String[] args) throws IOException {
        Path file = Path.of("LuckyNumber.csv");
        Random random = new Random();
        try (BufferedWriter writer = Files.newBufferedWriter(file, StandardCharsets.UTF_8)) {
            for (int i = 0; i < 500; i++) {
                writer.write(Integer.toString(random.nextInt(100) + 1));
                writer.newLine();
            }
        }

        System.out.println("Saved 500 numbers to " + file + ". Reading them back:");
        int count = 0;
        try (BufferedReader reader = Files.newBufferedReader(file, StandardCharsets.UTF_8)) {
            String line;
            while ((line = reader.readLine()) != null) {
                System.out.printf("%3d%s", Integer.parseInt(line), ++count % 20 == 0 ? "\n" : " ");
            }
        }
        System.out.println("Numbers read: " + count);
    }
}
```

Sample input and output

```
Command: java -cp build Exercise1

Program output:
Saved 500 numbers to LuckyNumber.csv. Reading them back:
 64  55  26  61  12  95 100  58  28  76  14  51  97   7  68  36  25  39  19  35
 97  48  55  59  66  10  77  14  44  88  55   7  18  68  77  28  97   4  56  85
 28   7  62  67  69  38  77  16   6  37  44  94 100   2  48  99   9  73  69  69
 13  55  77  27  62  69  94  77  88  95  43  24  47   3   5  58  54  60  16  41
 30  33  98  30  77  76  21  50  89  43  73  45  67  38  44  27  96  70  49  84
  2  90  60  68  98  68   3  60  79  38  55  12  24  64  43  86  34  79  83  56
 91   8   1   6  47  99  64  27  93  10  57  26   8   2  27  60  45   6  12  73
 56  71  22  25  41  16  96  26  80  50  57  60  59  88  59  28  73  26  77  42
 69   3  52  49   8  97  28  53  38  52  31  34  92  20  15   3  72  46  13  82
 92  77  11  35  64  18  45  84  30  97  40  44  59  35  12  59  76  86  92  15
 32  64  25  65  25  12  24  19  31  49  57  59  68  81  41  52  31  96  76   8
 56  51  68  71  32  43  62  91  66  54  82  18   1  92  42  52  42   7  56   9
 14  75  22  43  82  90  91  57  63 100  72  10  96  89  25  37  99  12  81  38
 64  32  86   5   5  73  86  58  55  38  28  81  98  27  54  36  88  11  10  92
 96  55  61  65  75  13  56  51  57  36  47   6  20  84  17  20  17  72   1  47
 40   1  58  66  59  80  70  81  44  60  58  13   8  37  14  61  75  35  60  96
 37 100  22  48  92  19  13  93  81   3  62  55  57  86  76  56  80   4   1   2
 29  69  37  21  20  10  81  66  42  66  81  65  28  96  23   9  30  29  89  78
 76  79  74  70  90  47  98   9  77  45  77  11  38  15  89  72  99   2  12  13
 34  94  71  38  25  79  14  44  96  76  38  88  30 100  13   9  27  20  86  51
 79   4  96  61  17  30  57  58  27  89  87  40  61  31  23  26  65  79  39  13
  5  15  49  42  32  41  99  50  52  24   2  78  50  98  95  10  17  65  53  58
 12  75  76  57  69  90  28  30  19  99  45  36  73  35  67  21  94  92  47  19
 29   7  93   4  73  33  94   8  49  44  21  85  55  12  97  63  44  95   6  82
 76  10  25  41   3   6  56  45   3  48   3  19  90  37  52  31  53  50  28  74
Numbers read: 500
```

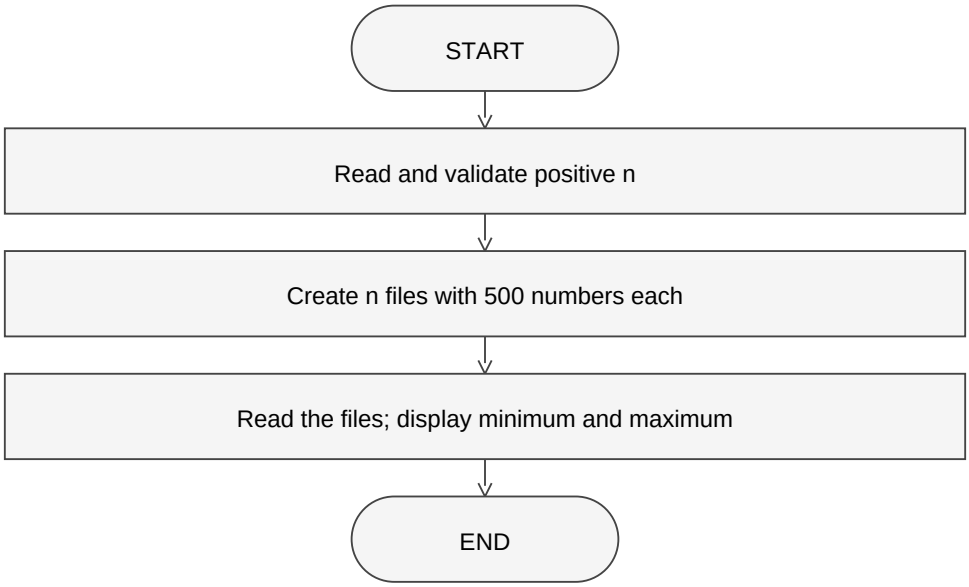
Exercise 2: Multiple files

Read 1,500 values from three files into an ArrayList.

Pseudocode

```
START
READ a positive file count n
FOR i = 1 TO n
    CREATE Ex2LuckyNumber/filei.csv
    WRITE 500 random integers from 1 to 100
END FOR
READ all n files into an ArrayList
DISPLAY the minimum and maximum
END
```

Flowchart



Exercise 2 - Source code

Exercise2.java

Exercise2.java

```java
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.Collections;
import java.util.Random;
import java.util.Scanner;

public class Exercise2 {
    private static final int NUMBERS_PER_FILE = 500;

    public static void main(String[] args) throws IOException {
        Scanner scanner = new Scanner(System.in, StandardCharsets.UTF_8);
        int n = ConsoleInput.readPositiveInt(scanner, "Number of files: ");
        Path folder = Path.of("Ex2LuckyNumber");
        Files.createDirectories(folder);
        Random random = new Random();
        for (int i = 1; i ≤ n; i++) {
            Path file = folder.resolve("file" + i + ".csv");
            try (BufferedWriter writer = Files.newBufferedWriter(file, StandardCharsets.UTF_8)) {
                for (int j = 0; j < NUMBERS_PER_FILE; j++) {
                    writer.write(Integer.toString(random.nextInt(100) + 1));
                    writer.newLine();
                }
            }
        }

        ArrayList<Integer> numbers = new ArrayList<>();
        for (int i = 1; i ≤ n; i++) {
            Path file = folder.resolve("file" + i + ".csv");
            try (BufferedReader reader = Files.newBufferedReader(file, StandardCharsets.UTF_8)) {
                String line;
                while ((line = reader.readLine()) != null) {
                    numbers.add(Integer.parseInt(line));
                }
            }
        }
        System.out.println("Files created: " + n);
        System.out.println("Numbers loaded into ArrayList: " + numbers.size());
        System.out.println("Minimum: " + Collections.min(numbers));
        System.out.println("Maximum: " + Collections.max(numbers));
    }
}
```

Sample input and output

```
Command: java -cp build Exercise2

Sample input:
3

Program output:
Number of files: Files created: 3
Numbers loaded into ArrayList: 1500
Minimum: 1
Maximum: 100
```

Exercise 3: Save students

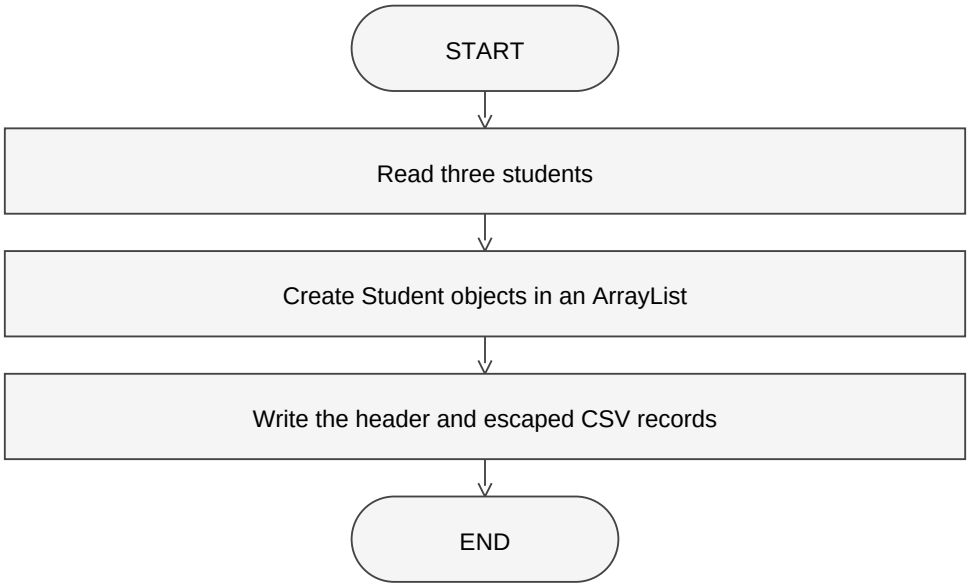
Enter three fictional demonstration team members.

The sample student details are fictional.

Pseudocode

```
START
CREATE an empty ArrayList of Students
REPEAT 3 times
    READ nonblank name, ID, and gender
    CREATE and ADD a Student object
END REPEAT
WRITE the header and escaped CSV records
SAVE Ex3StudentData.csv
END
```

Flowchart



Exercise 3 - Source code

Exercise3.java



Sample input and output

```
Command: java -cp build Exercise3

Sample input:
Zoe, Chan
001
female
Bora "Bo"
002
male
Ana
003
female

Program output:
Student 1 of 3
Name: ID: Gender: Student 2 of 3
Name: ID: Gender: Student 3 of 3
Name: ID: Gender: Saved 3 students to Ex3StudentData.csv
```

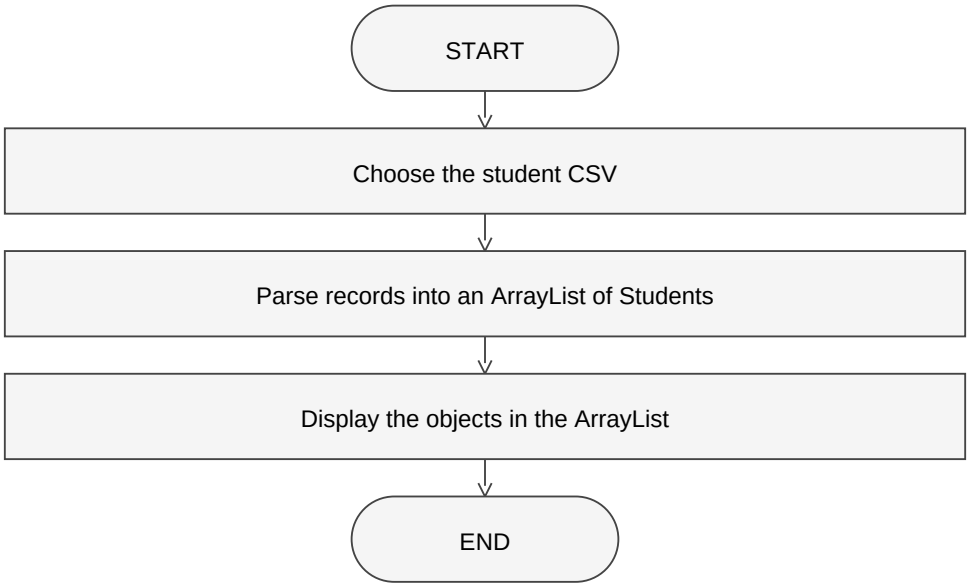
Exercise 4: Read students

Reconstruct students in an ArrayList and display them.

Pseudocode

```
START
SELECT StudentData.csv or Ex3StudentData.csv
READ and validate the CSV header
CREATE an empty ArrayList
FOR EACH student record
    PARSE fields and ADD a Student object
END FOR
DISPLAY each Student object
END
```

Flowchart



Exercise 4 - Source code

Exercise4.java

Exercise4.java

```java
import java.io.IOException;
import java.nio.file.Path;
import java.util.ArrayList;

public class Exercise4 {
    public static void main(String[] args) throws IOException {
        Path file = args.length > 0 ? Path.of(args[0]) : StudentData.defaultInput();
        ArrayList<Student> students = StudentData.read(file);
        System.out.println("Loaded " + students.size() + " students from " + file);
        for (Student student : students) {
            System.out.println(student);
        }
    }
}
```

Sample input and output

```
Command: java -cp build Exercise4

Program output:
Loaded 3 students from Ex3StudentData.csv
Name: Zoe, Chan | ID: 001 | Gender: female
Name: Bora "Bo" | ID: 002 | Gender: male
Name: Ana | ID: 003 | Gender: female
```

Exercise 5: Add and sort

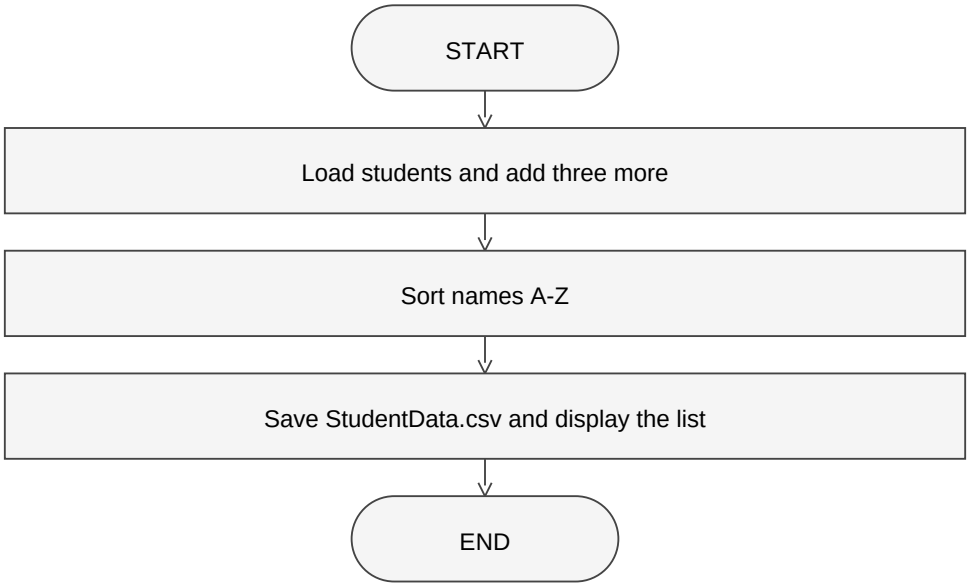
Add three demonstration students and save six names in A-Z order.

The sample student details are fictional.

Pseudocode

```
START
LOAD the saved students into an ArrayList
READ and ADD three more Students
SORT by name A-Z, ignoring case
WRITE the sorted list to StudentData.csv
DISPLAY the sorted list
END
```

Flowchart



Exercise 5 - Source code

Exercise5.java

Exercise5.java

```java
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.Scanner;

public class Exercise5 {
    public static void main(String[] args) throws IOException {
        Path input = args.length > 0 ? Path.of(args[0]) : StudentData.defaultInput();
        Path output = args.length > 1 ? Path.of(args[1]) : Path.of("StudentData.csv");
        ArrayList<Student> students = StudentData.read(input);
        System.out.println("Loaded " + students.size() + " students from " + input);
        Scanner scanner = new Scanner(System.in, StandardCharsets.UTF_8);
        students.addAll(StudentData.input(scanner, 3));
        students.sort(Comparator.comparing(Student::name, String.CASE_INSENSITIVE_ORDER)
                .thenComparing(Student::name).thenComparing(Student::id));
        StudentData.write(output, students);
        System.out.println("Saved " + students.size() + " students sorted by name (A-Z) to " + output);
        for (Student student : students) {
            System.out.println(student);
        }
    }
}
```

Sample input and output

```
Command: java -cp build Exercise5

Sample input:
dara
006
male
Chenda
005
female
Bopha
004
female

Program output:
Loaded 3 students from Ex3StudentData.csv
Student 1 of 3
Name: ID: Gender: Student 2 of 3
Name: ID: Gender: Student 3 of 3
Name: ID: Gender: Saved 6 students sorted by name (A-Z) to StudentData.csv
Name: Ana | ID: 003 | Gender: female
Name: Bopha | ID: 004 | Gender: female
Name: Bora "Bo" | ID: 002 | Gender: male
Name: Chenda | ID: 005 | Gender: female
Name: dara | ID: 006 | Gender: male
Name: Zoe, Chan | ID: 001 | Gender: female
```

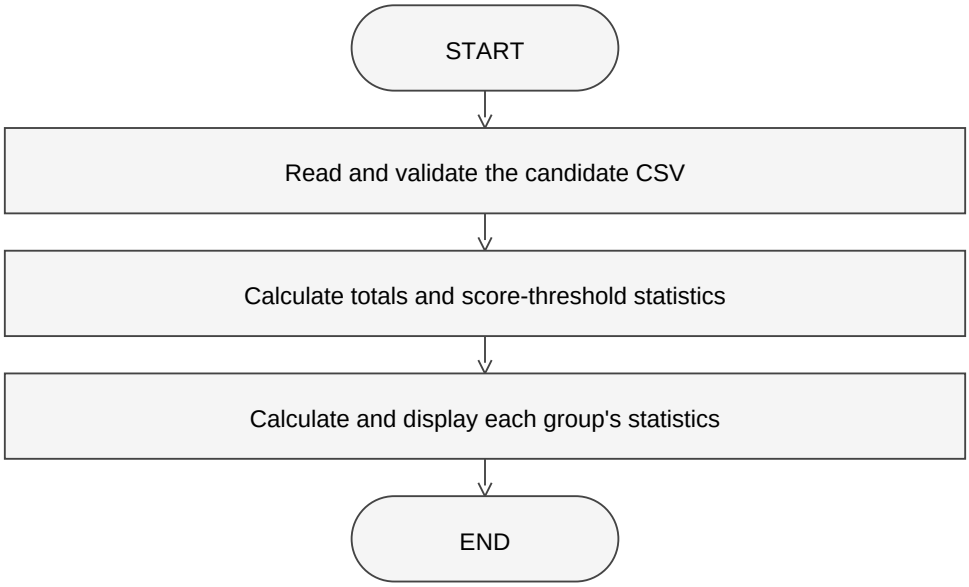
Exercise 6: Candidate analysis

Analyze the supplied local dataset; publish aggregate output only.

Pseudocode

```
START
READ the CSV and locate the actual header
PARSE and validate candidate records
COUNT all candidates and gender percentages
FILTER average scores >= 90 and summarize gender
COUNT scores >= 85 in each subject
COUNT candidates with all three scores >= 85
GROUP candidates and calculate group percentages
DISPLAY all answers
END
```

Flowchart



Exercise 6 - Source code

Exercise6.java

Exercise6.java

```java
import java.io.IOException;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Map;

public class Exercise6 {
    public static void main(String[] args) throws IOException {
        Path file = args.length > 0 && !args[0].equals("--summary")
                ? Path.of(args[0]) : CandidateData.DEFAULT_FILE;
        CandidateData data = CandidateData.read(file);
        if (!Arrays.asList(args).contains("--summary")) {
            data.display();
        }
        ArrayList<Candidate> candidates = data.candidates();
        CandidateStatistics.printGenderSummary("All candidates", candidates);
        CandidateStatistics.printGenderSummary("Average score ≥ 90",
                CandidateStatistics.averageAtLeast(candidates, 90));
        System.out.println("Math score ≥ 85: "
                + CandidateStatistics.subjectAtLeast(candidates, Candidate::math, 85));
        System.out.println("English score ≥ 85: "
                + CandidateStatistics.subjectAtLeast(candidates, Candidate::english, 85));
        System.out.println("Computer score ≥ 85: "
                + CandidateStatistics.subjectAtLeast(candidates, Candidate::computer, 85));
        System.out.println("All three subjects ≥ 85: "
                + CandidateStatistics.allSubjectsAtLeast(candidates, 85));
        System.out.println();
        for (Map.Entry<String, ArrayList<Candidate>> group
                : CandidateStatistics.byGroup(candidates).entrySet()) {
            CandidateStatistics.printGenderSummary("Group " + group.getKey(), group.getValue());
        }
    }
}
```

Sample input and output

```
Command: java -cp build Exercise6 data/CandidatePerformanceTest.csv --summary

Program output:
All candidates
Total candidates: 958
Male candidates: 467 (48.75%)
Female candidates: 491 (51.25%)

Average score >= 90
Total candidates: 48
Male candidates: 14 (29.17%)
Female candidates: 34 (70.83%)

Math score >= 85: 112
English score >= 85: 142
Computer score >= 85: 128
All three subjects >= 85: 58

Group A
Total candidates: 81
Male candidates: 50 (61.73%)
Female candidates: 31 (38.27%)

Group B
Total candidates: 185
Male candidates: 84 (45.41%)
Female candidates: 101 (54.59%)

Group C
Total candidates: 308
Male candidates: 134 (43.51%)
Female candidates: 174 (56.49%)

Group D
Total candidates: 249
Male candidates: 130 (52.21%)
Female candidates: 119 (47.79%)

Group E
Total candidates: 135
Male candidates: 69 (51.11%)
Female candidates: 66 (48.89%)
```

Exercise 7: Group exports

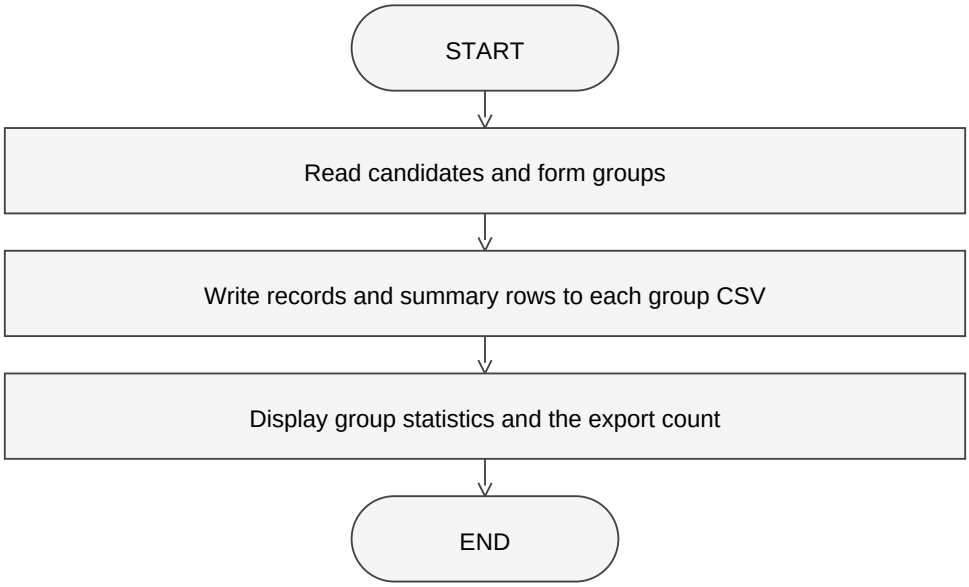
Export every group and append total and gender summaries.

The supplied dataset includes group E, so five files preserve all 958 candidates.

Pseudocode

```
START
READ candidates and GROUP them by group name
ENSURE groups A, B, C, and D exist
FOR EACH group, including group E when present
    WRITE the header and original candidate records
    APPEND total students and male/female percentages
    DISPLAY the group statistics
END FOR
DISPLAY the number of exported candidates
END
```

Flowchart



Exercise 7 - Source code

Exercise7.java

Exercise7.java

```java
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.List;
import java.util.Locale;
import java.util.Map;

public class Exercise7 {
    public static void main(String[] args) throws IOException {
        Path input = args.length > 0 && !args[0].equals("--summary")
                ? Path.of(args[0]) : CandidateData.DEFAULT_FILE;
        Path output = args.length > 1 && !args[1].equals("--summary")
                ? Path.of(args[1]) : Path.of("groups");
        CandidateData data = CandidateData.read(input);
        if (!Arrays.asList(args).contains("--summary")) {
            data.display();
        }
        Map<String, ArrayList<Candidate>> groups = CandidateStatistics.byGroup(data.candidates());
        for (String group : List.of("A", "B", "C", "D")) {
            groups.computeIfAbsent(group, key -> new ArrayList<>());
        }
        Files.createDirectories(output);
        int exported = 0;
        for (Map.Entry<String, ArrayList<Candidate>> entry : groups.entrySet()) {
            List<Candidate> members = entry.getValue();
            ArrayList<List<String>> rows = new ArrayList<>();
            rows.add(data.header());
            for (Candidate candidate : members) {
                rows.add(candidate.row());
            }
            int width = data.header().size();
            rows.add(summaryRow(width, "Summary - total students", Integer.toString(members.size())));
            rows.add(summaryRow(width, "Summary - male (%)", percent(members, "male")));
            rows.add(summaryRow(width, "Summary - female (%)", percent(members, "female")));
            Path file = output.resolve("group" + entry.getKey() + ".csv");
            Csv.write(file, rows);
            CandidateStatistics.printGenderSummary("Group " + entry.getKey(), members);
            System.out.println("Wrote " + file);
            System.out.println();
            exported += members.size();
        }
        System.out.println("Exported " + exported + " candidates to " + groups.size() + " group files.");
    }

    private static List<String> summaryRow(int width, String label, String value) {
        ArrayList<String> row = new ArrayList<>(Collections.nCopies(width, ""));
        row.set(0, label);
        row.set(1, value);
        return row;
    }

    private static String percent(List<Candidate> candidates, String gender) {
        return String.format(Locale.ROOT, "%.2f%%", CandidateStatistics.percentage(
                CandidateStatistics.countGender(candidates, gender), candidates.size()));
    }
}
```

Sample input and output

```
Command: java -cp build Exercise7 data/CandidatePerformanceTest.csv groups --summary

Program output:
Group A
Total candidates: 81
Male candidates: 50 (61.73%)
Female candidates: 31 (38.27%)

Wrote groups\groupA.csv

Group B
Total candidates: 185
Male candidates: 84 (45.41%)
Female candidates: 101 (54.59%)

Wrote groups\groupB.csv

Group C
Total candidates: 308
Male candidates: 134 (43.51%)
Female candidates: 174 (56.49%)

Wrote groups\groupC.csv

Group D
Total candidates: 249
Male candidates: 130 (52.21%)
Female candidates: 119 (47.79%)

Wrote groups\groupD.csv

Group E
Total candidates: 135
Male candidates: 69 (51.11%)
Female candidates: 66 (48.89%)

Wrote groups\groupE.csv

Exported 958 candidates to 5 group files.
```

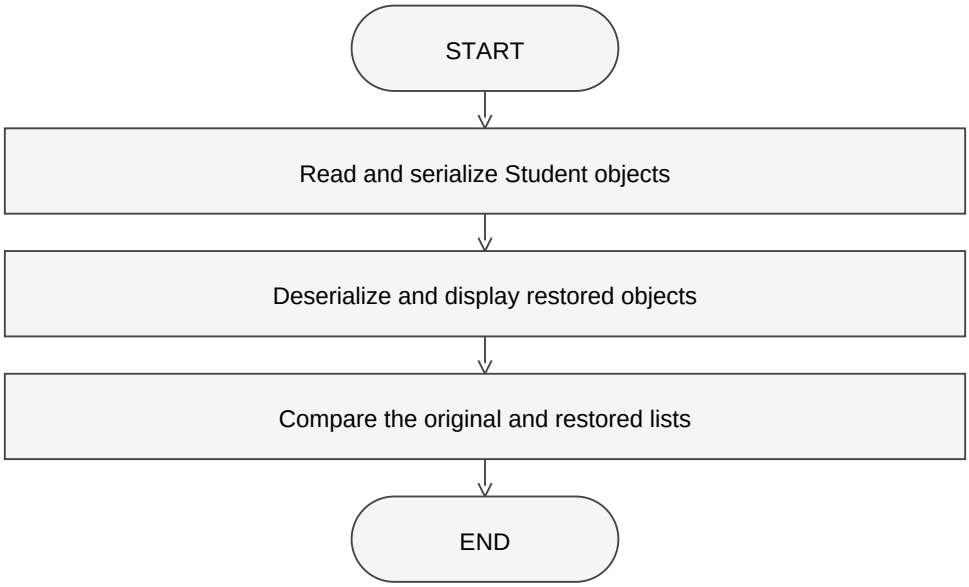
Bonus: CSV object round trip

Serialize Student objects and verify the reconstructed objects.

Pseudocode

```
START
LOAD the original Student objects
SERIALIZE the objects to BonusStudents.csv
DESERIALIZE the CSV into new Student objects
DISPLAY the restored objects
COMPARE the original and restored lists
IF equal THEN DISPLAY round-trip verified
ELSE REPORT a round-trip error
END
```

Flowchart



Bonus - Source code

BonusExercise.java

BonusExercise.java

```java
import java.io.IOException;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.List;

public class BonusExercise {
    public static void serialize(List<Student> students, Path file) throws IOException {
        StudentData.write(file, students);
    }

    public static ArrayList<Student> deserialize(Path file) throws IOException {
        return StudentData.read(file);
    }

    public static void main(String[] args) throws IOException {
        Path input = args.length > 0 ? Path.of(args[0]) : StudentData.defaultInput();
        Path output = args.length > 1 ? Path.of(args[1]) : Path.of("BonusStudents.csv");
        ArrayList<Student> original = StudentData.read(input);
        serialize(original, output);
        System.out.println("Serialized " + original.size() + " Student objects to " + output);
        ArrayList<Student> restored = deserialize(output);
        System.out.println("Deserialized " + restored.size() + " Student objects:");
        for (Student student : restored) {
            System.out.println(student);
        }
        if (!original.equals(restored)) {
            throw new IllegalStateException("CSV round-trip verification failed.");
        }
        System.out.println("Round-trip verified: every restored object equals its original.");
    }
}
```

Sample input and output

```
Command: java -cp build BonusExercise

Program output:
Serialized 6 Student objects to BonusStudents.csv
Deserialized 6 Student objects:
Name: Ana | ID: 003 | Gender: female
Name: Bopha | ID: 004 | Gender: female
Name: Bora "Bo" | ID: 002 | Gender: male
Name: Chenda | ID: 005 | Gender: female
Name: dara | ID: 006 | Gender: male
Name: Zoe, Chan | ID: 001 | Gender: female
Round-trip verified: every restored object equals its original.
```

Shared source code

ConsoleInput.java

ConsoleInput.java

```java
import java.util.Scanner;

public final class ConsoleInput {
    private ConsoleInput() { }

    public static String readRequired(Scanner scanner, String prompt) {
        while (true) {
            System.out.print(prompt);
            if (!scanner.hasNextLine()) {
                throw new IllegalStateException("Input ended before all required values were supplied.");
            }
            String value = scanner.nextLine().trim();
            if (!value.isEmpty()) {
                return value;
            }
            System.out.println("Please enter a non-empty value.");
        }
    }

    public static int readPositiveInt(Scanner scanner, String prompt) {
        while (true) {
            try {
                int value = Integer.parseInt(readRequired(scanner, prompt));
                if (value > 0) {
                    return value;
                }
            } catch (NumberFormatException exception) {
            }
            System.out.println("Please enter a positive integer.");
        }
    }
}
```

Shared source code

Csv.java

Csv.java

```java
import java.io.BufferedWriter;
import java.io.IOException;
import java.io.PushbackReader;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.List;

public final class Csv {
    private Csv() { }

    public static ArrayList<List<String>> read(Path file) throws IOException {
        ArrayList<List<String>> rows = new ArrayList<>();
        try (PushbackReader reader = new PushbackReader(
                Files.newBufferedReader(file, StandardCharsets.UTF_8), 1)) {
            ArrayList<String> row = new ArrayList<>();
            StringBuilder field = new StringBuilder();
            boolean quoted = false;
            boolean closedQuote = false;
            boolean first = true;
            int c;
            while ((c = reader.read()) != -1) {
                if (first && c == '\uFEFF') {
                    first = false;
                    continue;
                }
                first = false;
                if (quoted) {
                    if (c == '"') {
                        int next = reader.read();
                        if (next == '"') {
                            field.append('"');
                        } else {
                            quoted = false;
                            closedQuote = true;
                            if (next != -1) {
                                reader.unread(next);
                            }
                        }
                    } else {
                        field.append((char) c);
                    }
                } else if (c == ',' || c == '\r' || c == '\n') {
                    row.add(field.toString());
                    field.setLength(0);
                    closedQuote = false;
                    if (c != ',') {
```

Shared source code - continued

Csv.java

```
                        rows.add(row);
                        row = new ArrayList<>();
                        if (c == '\r') {
                            int next = reader.read();
                            if (next != '\n' && next != -1) {
                                reader.unread(next);
                            }
                        }
                    }
                } else if (c == '"' && field.length() == 0 && !closedQuote) {
                    quoted = true;
                } else {
                    if (closedQuote || c == '"') {
                        throw new IOException("Invalid quoting in CSV record " + (rows.size() + 1));
                    }
                    field.append((char) c);
                }
            }
            if (quoted) {
                throw new IOException("Unclosed quoted CSV field in " + file);
            }
            if (!row.isEmpty() || field.length() > 0 || closedQuote) {
                row.add(field.toString());
                rows.add(row);
            }
        }
        return rows;
    }

    public static void write(Path file, List<? extends List<String>> rows) throws IOException {
        try (BufferedWriter writer = Files.newBufferedWriter(file, StandardCharsets.UTF_8)) {
            for (List<String> row : rows) {
                for (int i = 0; i < row.size(); i++) {
                    if (i > 0) {
                        writer.write(',');
                    }
                    String value = row.get(i);
                    if (value.contains(",") || value.contains("\"")
                            || value.contains("\n") || value.contains("\r")) {
                        writer.write('"');
                        writer.write(value.replace("\"", "\"\""));
                        writer.write('"');
                    } else {
                        writer.write(value);
                    }
                }
                writer.newLine();
            }
        }
    }
}
```

Shared source code

Student.java



Shared source code

StudentData.java

StudentData.java

```java
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public final class StudentData {
    public static final List<String> HEADER = List.of("name", "ID", "gender");

    private StudentData() { }

    public static Path defaultInput() {
        Path canonical = Path.of("StudentData.csv");
        return Files.exists(canonical) ? canonical : Path.of("Ex3StudentData.csv");
    }

    public static ArrayList<Student> read(Path file) throws IOException {
        ArrayList<List<String>> rows = Csv.read(file);
        if (rows.isEmpty() || rows.get(0).size() != HEADER.size()) {
            throw new IOException("Expected CSV header: name,ID,gender in " + file);
        }
        for (int i = 0; i < HEADER.size(); i++) {
            if (!HEADER.get(i).equalsIgnoreCase(rows.get(0).get(i).trim())) {
                throw new IOException("Expected CSV header: name,ID,gender in " + file);
            }
        }
        ArrayList<Student> students = new ArrayList<>();
        for (int i = 1; i < rows.size(); i++) {
            List<String> row = rows.get(i);
            if (row.size() == 1 && row.get(0).isBlank()) {
                continue;
```

Shared source code - continued

StudentData.java

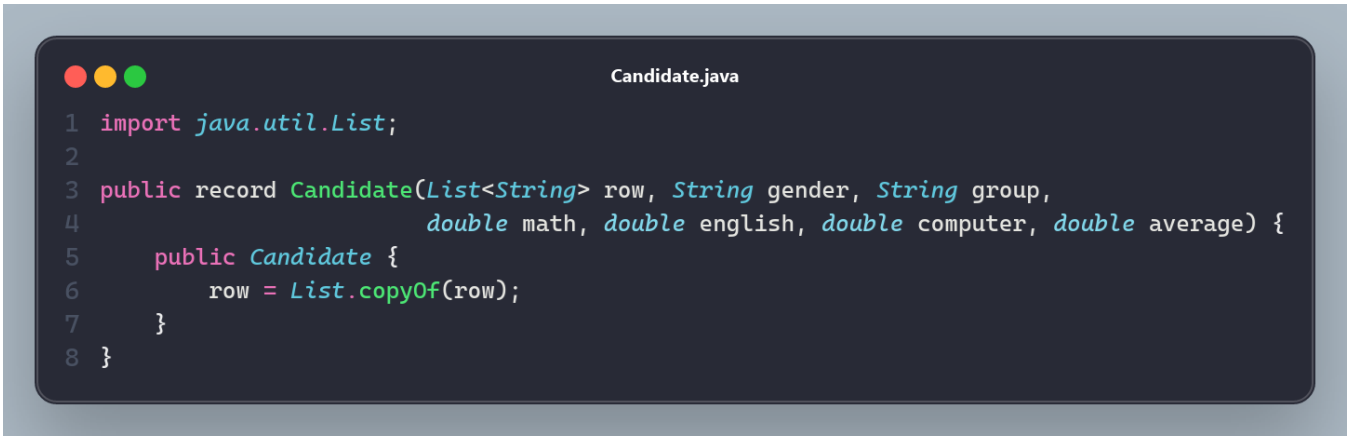
```
            }
            if (row.size() != 3) {
                throw new IOException("Expected three fields in student record " + (i + 1));
            }
            try {
                students.add(new Student(row.get(0), row.get(1), row.get(2)));
            } catch (IllegalArgumentException exception) {
                throw new IOException("Invalid student record " + (i + 1) + ": "
                        + exception.getMessage(), exception);
            }
        }
        return students;
    }

    public static ArrayList<Student> input(Scanner scanner, int count) {
        ArrayList<Student> students = new ArrayList<>();
        for (int i = 1; i ≤ count; i++) {
            System.out.println("Student " + i + " of " + count);
            String name = ConsoleInput.readRequired(scanner, "Name: ");
            String id = ConsoleInput.readRequired(scanner, "ID: ");
            String gender = ConsoleInput.readRequired(scanner, "Gender: ");
            students.add(new Student(name, id, gender));
        }
        return students;
    }

    public static void write(Path file, List<Student> students) throws IOException {
        ArrayList<List<String>> rows = new ArrayList<>();
        rows.add(HEADER);
        for (Student student : students) {
            rows.add(student.toRow());
        }
        Csv.write(file, rows);
    }
}
```

Shared source code

Candidate.java



Shared source code

CandidateData.java

CandidateData.java

```java
import java.io.IOException;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Locale;
import java.util.Map;

public record CandidateData(List<String> header, ArrayList<Candidate> candidates) {
    public static final Path DEFAULT_FILE = Path.of("data", "CandidatePerformanceTest.csv");
    private static final List<String> REQUIRED =
            List.of("id", "gender", "group", "math", "english", "computer", "average");

    public static CandidateData read(Path file) throws IOException {
        ArrayList<List<String>> rows = Csv.read(file);
        Map<String, Integer> columns = new HashMap<>();
        int headerIndex = -1;
        for (int i = 0; i < rows.size(); i++) {
            columns.clear();
            List<String> row = rows.get(i);
            for (int j = 0; j < row.size(); j++) {
                columns.put(normalize(row.get(j)), j);
            }
            if (columns.keySet().containsAll(REQUIRED)) {
                headerIndex = i;
                break;
            }
        }
        if (headerIndex < 0) {
            throw new IOException("Missing candidate header; required columns: " + REQUIRED);
        }
        List<String> header = List.copyOf(rows.get(headerIndex));
        ArrayList<Candidate> candidates = new ArrayList<>();
        for (int i = headerIndex + 1; i < rows.size(); i++) {
            List<String> row = rows.get(i);
            if (row.stream().allMatch(String::isBlank)) {
                continue;
            }
            try {
                if (row.size() != header.size()) {
                    throw new IllegalArgumentException("Expected " + header.size() + " fields");
```

Shared source code - continued

CandidateData.java

```
                }
                String gender = row.get(columns.get("gender")).trim().toLowerCase(Locale.ROOT);
                if (!gender.equals("male") && !gender.equals("female")) {
                    throw new IllegalArgumentException("Unknown gender: " + gender);
                }
                String group = row.get(columns.get("group")).trim().toUpperCase(Locale.ROOT);
                if (group.matches("GROUP\\s+[A-Z]")) {
                    group = group.substring(group.length() - 1);
                }
                if (!group.matches("[A-Z]")) {
                    throw new IllegalArgumentException("Invalid group: " + group);
                }
                candidates.add(new Candidate(row, gender, group,
                        score(row, columns, "math"), score(row, columns, "english"),
                        score(row, columns, "computer"), score(row, columns, "average")));
            } catch (IllegalArgumentException exception) {
                throw new IOException("Invalid candidate CSV record " + (i + 1) + ": "
                        + exception.getMessage(), exception);
            }
        }
        return new CandidateData(header, candidates);
    }

    private static String normalize(String value) {
        return value.trim().toLowerCase(Locale.ROOT).replaceAll("[^a-z0-9]", "");
    }

    private static double score(List<String> row, Map<String, Integer> columns, String name) {
        double value = Double.parseDouble(row.get(columns.get(name)).trim());
        if (!Double.isFinite(value) || value < 0 || value > 100) {
            throw new IllegalArgumentException(name + " score must be between 0 and 100");
        }
        return value;
    }

    public void display() {
        System.out.println(String.join(" | ", header).replace('\r', ' ').replace('\n', ' '));
        for (Candidate candidate : candidates) {
            System.out.println(String.join(" | ", candidate.row()));
        }
        System.out.println();
    }
}
```

Shared source code

CandidateStatistics.java

CandidateStatistics.java

```java
import java.util.ArrayList;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.TreeMap;
import java.util.function.ToDoubleFunction;

public final class CandidateStatistics {
    private CandidateStatistics() { }

    public static long countGender(List<Candidate> candidates, String gender) {
        return candidates.stream().filter(candidate -> candidate.gender().equals(gender)).count();
    }

    public static double percentage(long count, int total) {
        return total == 0 ? 0.0 : 100.0 * count / total;
    }

    public static ArrayList<Candidate> averageAtLeast(List<Candidate> candidates, double minimum) {
        ArrayList<Candidate> result = new ArrayList<>();
        for (Candidate candidate : candidates) {
            if (candidate.average() ≥ minimum) {
                result.add(candidate);
            }
        }
        return result;
    }

    public static long subjectAtLeast(List<Candidate> candidates,
                                      ToDoubleFunction<Candidate> score, double minimum) {
        return candidates.stream().filter(candidate -> score.applyAsDouble(candidate) ≥ minimum).count();
    }

    public static long allSubjectsAtLeast(List<Candidate> candidates, double minimum) {
        return candidates.stream().filter(candidate -> candidate.math() ≥ minimum
                && candidate.english() ≥ minimum && candidate.computer() ≥ minimum).count();
    }

    public static Map<String, ArrayList<Candidate>> byGroup(List<Candidate> candidates) {
        Map<String, ArrayList<Candidate>> groups = new TreeMap<>();
        for (Candidate candidate : candidates) {
            groups.computeIfAbsent(candidate.group(), key -> new ArrayList<>()).add(candidate);
        }
        return groups;
    }

    public static void printGenderSummary(String title, List<Candidate> candidates) {
        System.out.println(title);
        System.out.println("Total candidates: " + candidates.size());
        long male = countGender(candidates, "male");
        long female = countGender(candidates, "female");
        System.out.printf(Locale.ROOT, "Male candidates: %d (%.2f%%)%n", male,
                percentage(male, candidates.size()));
        System.out.printf(Locale.ROOT, "Female candidates: %d (%.2f%%)%n%n", female,
                percentage(female, candidates.size()));
    }
}
```

Verification

Compiled for Java 17 and checked against independent CSV calculations. Candidate records and group exports stay local. The report contains aggregate candidate results.

Code images were generated with the CodeSnap RS 0.3.0 adapter from anupheap/codesnap-rs-vscode, installed in VS Code.

```
PASS Exercise 1: 500 saved numbers match the displayed file contents
PASS Exercise 2: file counts, input validation, stale files, and exact extrema
PASS Exercise 3: three students, required input, CSV escaping, and leading-zero IDs
PASS Exercise 4: ArrayList display, file-name fallback, BOM, and multiline fields
PASS Exercise 5: append exactly three, sort, save, and reread six students
PASS original local dataset: 958 candidates (not published)
PASS Exercise 6: threshold boundaries, groups, display, and empty populations
PASS Exercise 7: all groups, lossless partition, rectangular CSV, footer statistics, empty groups
PASS Bonus: object round trips preserve escaped fields, multiline names, and text IDs
```

Git repository: https://github.com/anupheap/PDI-Lab4